# How to Configure Git with Multiple User Credentials

Because sometimes, one Git identity just doesn't cut it

Many engineers work across multiple Git accounts: a personal GitHub profile, a company GitHub Enterprise tenancy, perhaps a customer's GitLab instance, or an open-source project elsewhere. Each of these accounts will require its own authentication, but without additional config, managing and accessing these repos can be a headache.

Instead of having to manually add credentials for every repository or on every Git CLI command, there's a better way.

Fortunately, Git provides a way to manage multiple credentials cleanly by using **credential namespaces** and the **Git Credential Manager (GCM)**.

## Why This Matters

If you use the same Git client for both personal and professional projects, you'll often hit friction points:

- Authentication popups are asking for the wrong account.
- Commits are authored under the wrong email address.
- Confusion about which credential Git is actually using.

By configuring **namespaces**, you can isolate credentials and identities per project. This keeps your Git setup predictable, clean, and less error-prone, especially if you use HTTPS authentication (as most developers do).

What's more, you can set a default namespace if you use one set of creds a lot more, and you can set up each git repo with its own namespace.

## How It Works

Git stores authentication details through a **credential helper**.
In our case, we'll use the **Git Credential Manager (GCM)**, a secure, cross-platform helper maintained by Microsoft and built into modern Git installations.

If you don't already have GCM, here's the install instructions:
https://github.com/git-ecosystem/git-credential-manager/blob/release/docs/install.md

By combining this with the `credential.namespace` setting, we can store multiple, separate credentials and tell Git which one to use for each repository.

## Step 1 a— Configure Your Personal Credentials

Run these commands to create a **personal** namespace and store your credentials for your personal Git account:

```
git config --global credential.personal.namespace personal
git config --global credential.personal.name <Personal Account Username>
git config --global credential.personal.email <Personal Account Email>
git config --global credential.personal.helper manager
```

What these do:

- `credential.personal.namespace` defines a unique name for this credential set (“personal” in this case).
- `credential.personal.name` and `.email` store your personal identity for commits.
- `credential.personal.helper manager` tells Git to use **Git Credential Manager** to securely store and recall tokens for this namespace.

## Step 1b— Configure Your Work Credentials

Now, create a second set for your work account:

```
git config --global credential.work.namespace work
git config --global credential.work.name <Work Account Username>
git config --global credential.work.email <Work Account Email>
git config --global credential.work.helper manager
```

You can replace `work` with another label if you have multiple tenancies (e.g. `clientA`, `opensource`, etc.).

If you happen to use the same username and email for both accounts, you can also add them globally so that Git defaults to them when unspecified:

```
git config --global user.name <username>
git config --global user.email <email>
```

Repeat step *1b* for any additional credentials you need to set up

## Step 2 — (Optional) Configure a Default Namespace

If you usually use the same credentials for most of your work, you don't want to have to configure every repo individually. You can set a global namespace, which git will default to, unless you tell it otherwise (see Step 3 for that).

```
git config --global credential.namespace personal
```

## Step 3 — Tell Each Repository Which Account to Use

Every local repository needs to know which credential namespace to use. If you completed Step 2, then repos will inherit the global setting. However, you will need to override this for any repos which use a different credential.

Here's how:

🙋 *tip: if you haven't cloned the repo yet, see Step 5.*

Inside the repo directory, run:

```
git config credential.namespace <Account Name>
```

For example:

```
git config credential.namespace personal
```

or:

```
git config credential.namespace work
```

This ensures Git fetches the correct credentials from the right namespace when authenticating.

If you want to override the commit author name or email in this repository, you can add:

```
git config user.name <username>
git config user.email <email>
```

⚠️ *If you use commit signing, then you will need to ensure that the email associated with your signing key is the same as the email in your git credentials, so make sure you set the correct email in each repo.*

## Step 4 — Authenticate Each Namespace Once

The first time you push to a remote in each namespace, Git will prompt you to log in (this is triggered by GCM).

- Push from a repo configured with `personal` → log in with your **personal** GitHub account.
- Push from a repo configured with `work` → log in with your **work** GitHub (or GitLab, Bitbucket, etc.) account.

Each login is stored securely in your credential manager under its own namespace. From then on, Git knows which credential to use for each repo — no more manual sign-ins or crossovers.

## Step 5 — Cloning with a Namespace

When cloning new repositories, you can also specify which credential namespace to use right away:

```
git clone -c credential.namespace=personal https://github.com/<username>/<repo>.
```

This ensures the repository is linked to the correct credentials from the start.

## How Credential Namespaces Help

Once you've completed these steps, Git will maintain separate login tokens for each namespace.

You can view or manage these credentials through your system's credential manager (Windows, macOS Keychain, or Linux Secret Service).

It's a simple configuration, but it prevents a lot of common mistakes, especially for developers who switch between different accounts throughout the day.

## Quick Recap

- **Namespace** — defines an isolated identity and credential store.
- **Name / Email** — identifies who made the commits in that namespace.
- **Helper** — defines which credential manager to use (we use `manager`, i.e. Git Credential Manager).
- **Repo Config** — tells Git which namespace to use for that specific codebase.
- **Credential Manager** — securely stores and recalls authentication tokens for each namespace.

Git | Git Credential Manager | Source Control | Authentication | Github

## Published in on:tech

17 followers · Last published Feb 2, 2026

Follow

Brought to you by Leighton, on:tech publishes expert insights on AWS, app development, software development and technology.

## Written by Greg Farrow

825 followers · 84 following

Follow

Principal Engineer. I 🧡 Serverless

## No responses yet

Write a response

What are your thoughts?

Geg Farrow

on:tech by Ronnie R. Winter

## Recommended from Medium

Choose design patterns based on pain points: apply the right pattern with minimal over-...